

Structural Patterns & Advanced OOP Architecture

Adapter Pattern

Intent: Allows incompatible interfaces to work together. Acts as a bridge between two incompatible interfaces.

Real-World Analogy

A travel power adapter that lets you plug a US device into a European socket - same function, different interface.

When to Use

- When you want to use an existing class but its interface doesn't match what you need
- When creating reusable classes that need to work with unpredictable interfaces

Adapter Pattern - Code Walkthrough

The `SocketAdapter` wraps `EuropeanPlug` and exposes the `IUSASocket` interface, allowing the `Laptop` client to work seamlessly with an incompatible plug type.

```
// Existing class with incompatible interface
public class EuropeanPlug
{
    public string GetEuropeanSocket()
    {
        return "European socket";
    }
}

// Target interface expected by client
public interface IUSASocket
{
    string GetUSASocket();
}

// Adapter class
public class SocketAdapter: IUSASocket
{
    private readonly EuropeanPlug _europeanPlug;

    public SocketAdapter(EuropeanPlug europeanPlug)
    {
        _europeanPlug = europeanPlug;
    }

    public string GetUSASocket()
    {
        // Adapts European socket to USA format
        return $"Adapter converting: {_europeanPlug.GetEuropeanSocket()}";
    }
}

// Client code
public class Laptop
{
    public void Charge(IUSASocket socket)
    {
        Console.WriteLine($"Charging using {socket.GetUSASocket()}");
    }
}

// Usage
var laptop = new Laptop();
var europeanPlug = new EuropeanPlug();
var adapter = new SocketAdapter(europeanPlug);
laptop.Charge(adapter);
```

Decorator Pattern

Intent: Adds new functionality to objects dynamically without altering their structure. Follows the Open/Closed Principle - open for extension, closed for modification.

Real-World Analogy

Adding toppings to a pizza - the base pizza remains the same, but you wrap it with additional ingredients to enhance it.

When to Use

- When you need to add responsibilities **dynamically and transparently**
- When extension by subclassing is impractical or leads to class explosion
- When you want to avoid rigid inheritance hierarchies

Decorator Pattern - Code Walkthrough

Each decorator wraps an `ICoffee` component, adding behavior (cost, description) without modifying the original class. Decorators can be stacked freely at runtime.

```
// Component interface
public interface ICoffee
{
    string GetDescription();
    double GetCost();
}

// Concrete component
public class SimpleCoffee: ICoffee
{
    public string GetDescription() =>"Simple coffee";
    public double GetCost() =>10.00;
}

// Base decorator
public abstract class CoffeeDecorator: ICoffee
{
    protected readonly ICoffee _coffee;
    protected CoffeeDecorator(ICoffee coffee)
    {
        _coffee = coffee;
    }

    public virtual string GetDescription() =>_coffee.GetDescription();
    public virtual double GetCost() =>_coffee.GetCost();
}

// Concrete decorators
public class MilkDecorator: CoffeeDecorator
{
    public MilkDecorator(ICoffee coffee) : base(coffee) {}
    public override string GetDescription() =>_coffee.GetDescription() + ", Milk";
    public override double GetCost() =>_coffee.GetCost() + 2.00;
}

public class SugarDecorator: CoffeeDecorator
{
    public SugarDecorator(ICoffee coffee) : base(coffee) {}
    public override string GetDescription() =>_coffee.GetDescription() + ", Sugar";
    public override double GetCost() =>_coffee.GetCost() + 0.50;
}

// Usage
ICoffee coffee = new SimpleCoffee();
coffee = new MilkDecorator(coffee);
coffee = new SugarDecorator(coffee);
Console.WriteLine($"{coffee.GetDescription()} - ${coffee.GetCost()}");
// Output: Simple coffee, Milk, Sugar - $12.50
```

Repository Pattern

Intent: Abstracts data access logic, providing a clean separation between business logic and data access layers.



Centralized Logic

All data access is consolidated in one place, making it easier to maintain and audit.



Easier Unit Testing

Mock repositories can be injected, enabling isolated tests without a real database.



Decoupled Data Sources

Swap between SQL, MongoDB, or APIs without touching business logic.



Single Responsibility

Each repository has one job - managing persistence for a specific entity type.

Repository Pattern - Code Walkthrough

The `IProductRepository` interface defines the contract. `ProductRepository` implements it with Entity Framework.

`ProductService` depends only on the abstraction - never the concrete data layer.

```
// Domain entity
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}

// Repository interface (abstraction)
public interface IProductRepository
{
    Task<Product> GetByIdAsync(int id);
    Task<IEnumerable<Product>> GetAllAsync();
    Task<Product> AddAsync(Product product);
    Task UpdateAsync(Product product);
    Task DeleteAsync(int id);
}

// Concrete implementation
public class ProductRepository: IProductRepository
{
    private readonly AppDbContext _context;
    public ProductRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<Product> GetByIdAsync(int id) =>
        await _context.Products.FindAsync(id);

    public async Task<IEnumerable<Product>> GetAllAsync() =>
        await _context.Products.ToListAsync();

    public async Task<Product> AddAsync(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
        return product;
    }

    // UpdateAsync and DeleteAsync follows same pattern...
}

// Service layer using repository
public class ProductService
{
    private readonly IProductRepository _repository;
    public ProductService(IProductRepository repository)
    {
        _repository = repository;
    }

    public async Task<Product> CreateProductAsync(string name, decimal price)
    {
        var product = new Product
        {
            Name = name,
            Price = price
        };
        return await _repository.AddAsync(product);
    }
}
```

Asynchronous Programming - Why `async/await`?

The Problem

Blocking threads for I/O operations **wastes resources** and kills scalability. A blocked thread cannot serve other requests.

The Solution

`async/await` enables non-blocking asynchronous programming that *looks* like synchronous code - clean and readable.

The thread is freed during I/O waits, allowing the thread pool to handle other requests - dramatically improving throughput under load.

Key Concepts

- `async` marks a method as asynchronous
- `await` performs a non-blocking wait for a task to complete
- Methods return `Task` or `Task<T>`
- The thread is **released back to the pool** during I/O wait

async/await - Code Examples

Basic Async & Error Handling

```
// Basic async method
public async Task<string> DownloadDataAsync(string url)
{
    using var client = new HttpClient();
    // Thread released here while waiting
    var result = await client.GetStringAsync(url);
    return result;
}
```

Parallel Execution

```
// Parallel execution (when tasks are independent)
public async Task ProcessMultipleAsync()
{
    var task1 = FetchDataAsync(); // Starts immediately
    var task2 = FetchOtherAsync(); // Starts immediately
    var results = await Task.WhenAll(task1, task2);
}
```

Anti-Patterns

```
// Anti-pattern: async void (BAD)
public async void BadAsync() {} // Avoid!
// Good: Always return Task
public async Task GoodAsync() {}
```

async/await - Best Practices

✓ DO

- Return `Task` or `Task<T>` instead of `void`
- Name async methods with `Async` suffix (`GetDataAsync`)
- Use `await` - prevents deadlocks!

✗ DON'T

- Use `async void` except for event handlers
- Mix synchronous and asynchronous code unnecessarily